Celeste-RL: Comparison of Reinforcement Learning Methods on a Precision Platformer

State Representation Dominates Algorithm Choice

Jorge Manuel Torre
jmt1006@vt.edu
Virginia Tech
Blacksburg, VA, USA

Akhil Madipalli
akhimadi@vt.edu
Virginia Tech
Blacksburg, VA, USA

Maryam Malik
maryam23@vt.edu
Virginia Tech
Blacksburg, VA, USA

Figure 1: *Celeste Classic* runs on the PICO-8 fantasy console. Madeline (right) climbs a one-screen room before transitioning to the next.

ABSTRACT

For our final project in Machine Learning (CS 4824/ECE 4424), we compared five reinforcement learning methods: plain DQN, Dueling DQN with a counts-based curiosity bonus, Behavioral Cloning (BC), Hybrid (BC + DQN), and a six-stage spawn curriculum on the first room of *Celeste Classic*, a frame-perfect 2D platformer with rewards occurring at the end of every level. All methods share the same Gymnasium-compatible environment, action space, reward function, and hyperparameters where applicable. We found that **state representation dominates algorithm choice** as the deciding factor. Replacing the network's input from raw PICO-8 tile IDs (0-255) with five-class semantic encoding (air, solid, spike, out-of-bounds sentinel, other) raised plain DQN's completion rate from approximately 8% to 57-68% on the same algorithm. Across all five method variants, every "enhancement" relative to plain DQN with the semantic encoding underperforms or matches it. Four of them (BC, Hybrid, Curriculum, raw-IDs DQN) achieve 0% deterministic-equivalent completion. We position our work as an extension of two prior public RL attempts on the Pyleste emulator, both of which used purely scalar state representations and did not address the perception question.

KEYWORDS

reinforcement learning, deep Q-networks, behavioral cloning, curriculum learning, state representation, precision platformers, Celeste Classic, PICO-8

1 INTRODUCTION

Celeste Classic is a compact platform game originally written for the PICO-8 fantasy console. Its first room appears mechanically simple, where a player must climb to the top of a single screen, exiting when the in-game y-coordinate crosses below -4. However, successful completion requires a frame-perfect wall-jump-and-dash sequence at the upper ledge, which is known to hinder value-based reinforcement learning methods. The room exhibits three properties that stress-test RL approaches:

(1) **Extended decision window.** The agent must execute 50-100 correct decisions before any terminal reward is observed.
(2) **Frame-perfect input execution requirements.** The final dash-jump combination at the upper ledge spans roughly 5-10 frames during which each input must be precise; even a single random action breaks the sequence.
(3) **Sparse terminal reward.** The +500 completion bonus is observed almost never under purely random or ε-greedy exploration.

These properties make Celeste's first room a useful benchmark for isolating which RL components contribute usefully to a precision-control task.

We observe the following:

- A controlled comparison of five RL methods in a shared environment, action space, and reward function, evaluated under a single ε=0.05 per 100-episode protocol (Section 5).
- An ablation study showing that **state representation dominates algorithm choice** as the deciding factor: replacing raw PICO-8 tile IDs with a five-class semantic tile encoding raises plain DQN's completion rate from 8% to 57-68% (Section 6.1).
- Negative results on four DQN-family enhancements: Dueling architecture, counts-based curiosity bonus, DQN & BC Hybrid, and spawn-curriculum learning all underperform or match plain DQN with the same state encoding (Section 6).
- A reproducible artifact comprising the codebase, pretrained checkpoints, batched evaluation script, comparison plots, and a debugging devlog covering nine iteration phases[1].

Two prior public RL attempts on the Pyleste emulator predate this work: Salies/RLeste [10], a stable-baselines3 DQN with a six-dimensional scalar state, and dav1dm0/PPO-Agent-for-Pyleste [5], a PPO agent with a four-dimensional scalar state. Both used purely scalar (4-6 dim) state representations consisting only of player kinematics and neither addressed state representation as a factor, or

[1] https://github.com/jmtorr3/celeste-rl

ran a controlled multi-method comparison. Our work extends both with (a) a higher-dimensional, perception-driven state encoding, and (b) a controlled comparison spanning five algorithms.

2 RELATED WORKS

Our work draws on several lines of prior research.

Deep Q-Networks. The DQN algorithm of Mnih et al. [8] is the foundation of our four DQN-family variants. We additionally implement Double DQN [12] (policy-network action selection with target-network value evaluation) to reduce Q-value overestimation. The Dueling DQN architecture [13], which decomposes $Q(s, a)$ into a value stream $V(s)$ and an advantage stream $A(s, a)$, serves as the architectural variant in our comparison.

Intrinsic motivation. Our curiosity-bonus variant uses the counts-based intrinsic motivation framework of Bellemare et al. [1], applied as a +0.5 reward for each newly-visited 4-pixel cell within an episode and an additional +0.2 for cells visited fewer than ten times globally.

Imitation learning and hybrid methods. Our BC implementation is a standard cross-entropy classifier, susceptible to the well-known compounding-error problem of distribution-shifted imitation policies [9]. Our Hybrid implementation pre-loads a Tool-Assisted Speedrun (TAS) trajectory into a protected partition of the replay buffer, providing it with a warm-start akin to Deep Q-Learning from Demonstrations (DQfD) [7], though with a simpler partition strategy.

Curriculum learning. The backwards-spawn curriculum follows the original curriculum-learning framework of Bengio et al. [2], where the agent is first exposed to easier instances of the level (stages begin closer to the exit) before setting the goalpost back after each stage. This eventually leads to the agent encountering the full level.

Precision-control RL.. Outside DQN, on-policy methods with intrinsic motivation, such as PPO [11] with Random Network Distillation [3] are well-documented to perform better in sparse-reward, temporally-distant settings. We do not evaluate these methods here but instead suggest them as future work.

Celeste Classic agents. Two prior public RL agents on the Pyleste emulator are RLeste [10] and PPO-Agent-for-Pyleste [5], discussed in Section 1. Outside the Pyleste ecosystem, Celeste-Bot [6] solves the full game using a genetic algorithm against a C#/Unity reimplementation of *Celeste Classic*. The GA approach sidesteps the credit-assignment and exploration-noise issues that bottleneck DQN-family methods on this task.

3 ENVIRONMENT

We wrapped the Pyleste [4] Python emulator in a Gymnasium-compatible environment. A single environment step advances Pyleste by one game frame. Episodes end on death (player object removed without room transition), level completion (player object removed concurrent with a `player_spawn` appearing), or after 500 steps (truncation).

3.1 State representation

The state vector has 87 dimensions per frame:

- Six scalar features: ($x/128$, $y/128$, $v_x/4$, $v_y/4$, grace/6, djump) where *grace* is the coyote-time grace period and *djump* is dash availability.
- A $9 \times 9 = 81$-dimensional tile grid centered on the player position. Each tile is mapped to one of five semantic classes listed in the table below.

Table 1: Semantic tile encoding. The numerical value is what the network receives; the original PICO-8 tile ID space (0–255) is mapped to five categories.

Class	Value	Meaning
out-of-bounds	−2.0	sentinel for tiles outside the room
spike	−1.0	death tiles (IDs 17, 27, 43, 59)
air	0.0	empty space
other	0.5	decoration, fruit
solid	1.0	platform or wall

Semantic encoding is the most consequential design choice in this work, and is examined as an ablation study in Section 6.1. An earlier iteration of this project passed raw PICO-8 tile IDs (0–255) directly to the network, where a representation under which a spike (ID 17) and a decoration (ID 18) are numerically as similar as air (0) and a solid wall (1).

3.2 Action space

To remove redundancies, We use a 15-action discrete subset of the full 64-button-combination space:

$$\mathcal{A} = \{\text{idle}, L, R, J, J{+}L, J{+}R, D_{\theta_1}, \ldots, D_{\theta_8}\}$$

where J denotes jump and D_{θ_i} denotes a dash in one of eight directions.

3.3 Reward function

Reward shaping consists of:

- 15-step milestone ramp from $y = 50$ to $y = -5$, awarding one-time bonuses per-episode ranging from 10 (at $y < 50$) to 900 (at $y < -5$).
- Small movement bonus (+0.01 per frame the player has moved) and a stuck penalty (−0.1 after 30 stationary frames).
- Terminal reward of +500 on level completion and −5 on death.

The variant V_3 (TRAIN_V3) also adds a Bellemare-style counts-based exploration bonus, described in Section 4.

4 METHODS

We evaluated five methods plus a random-action baseline. All methods share the environment, state representation, action space, and reward function described in Section 3, and (where applicable) the same hyperparameters: learning rate $5{\times}10^{-4}$, $\gamma = 0.99$, batch size 128, replay buffer 500,000, ε start 1.0 decaying to 0.05 at rate 0.999990 per episode.

(1) Random baseline. Uniform sampling over the 15-action space at every step.

(2) Plain DQN.. Standard DQN with Double Q-learning. Network architecture: Input(87) → Dense(256) → Dense256 → Dense128 → Output(15) MLP with ReLU activations.

(3) Dueling DQN with curiosity bonus. The same network's final layers are split into a value stream $V(s) \in \mathbb{R}$ and an advantage stream $A(s,a) \in \mathbb{R}^{15}$, combined as

$$Q(s,a) = V(s) + \left(A(s,a) - \tfrac{1}{|\mathcal{A}|}\sum_{a'} A(s,a')\right).$$

The reward function is augmented with a counts-based intrinsic motivation term [1]: +0.5 for each newly-visited $(\lfloor x/4 \rfloor, \lfloor y/4 \rfloor)$ cell within an episode, +0.2 for cells visited fewer than ten times.

(4) Behavioral Cloning (BC).. Supervised classification on 66 (state, action) transitions extracted by replaying a TAS any% solution through the environment. Action-weighted cross-entropy loss with Gaussian noise ($\sigma = 0.02$) on the kinematic features (positions and velocities) to mitigate overfitting on the exact trajectory, trained for 200 epochs.

(5) Hybrid (BC + DQN).. A DQN agent whose replay buffer is partitioned 20%/80% between expert (TAS) and online experience. Expert transitions are pre-loaded and never evicted; online transitions are FIFO. The agent samples 20% of each batch from the expert partition.

(6) Curriculum learning. A six-stage backwards spawn schedule (Table 2). The agent advances from stage i to stage $i+1$ when it achieves $\geq 50\%$ completion rate over the most recent 100 episodes. If 2,000 episodes elapse without advancement, the stage times out. Network weights and replay buffer are carried between stages.

Table 2: Curriculum spawn schedule. Lower spawn y corresponds to a position farther from the exit (the room exit is at $y < -4$).

Stage	Spawn y	Max steps
1	15	100
2	30	150
3	50	200
4	60	250
5	71	300
6	96 (full level)	500

5 EXPERIMENTS

5.1 Training protocol

Each method is trained for 5,000 episodes (or until convergence as defined by per-method criteria). Training is performed on a single Blackwell GB10 GPU. Across the full project (including failed and superseded runs), we executed approximately 3×10^4 training episodes.

Table 3: Final comparison: completion rate under ε=0.05, 100-episode evaluation. The 57-68% range for `dqn_r1` reflects single-seed variance between an isolated 100-episode evaluation (68%) and a batched re-run on the same checkpoint (57%); the qualitative ordering is stable.

Method	State / Architecture	Completion
Plain DQN	semantic, plain DQN	**57–68%**
Dueling + curiosity	semantic, Dueling DQN	37-49%
Random	—	0%
Behavioral Cloning	semantic, plain DQN	0%
Hybrid (BC + DQN)	semantic, plain DQN	0%
Curriculum (6-stage)	semantic, Dueling DQN	0%
DQN, raw tile IDs	raw, Dueling DQN	0% (8% peak)

5.2 Evaluation protocol

We examined each trained checkpoint by running 100 episodes from the room's normal spawn position ($y = 96$) at $\varepsilon = 0.05$. The $\varepsilon = 0.05$ matches the noise level the policy was trained against. Pure greedy evaluation ($\varepsilon = 0$) led to identical-trajectory results, as deterministic argmax behavior across all methods prevented any meaningful exploration of the state space. We classified each episode's outcome into one of three categories:

- **Complete**: player crosses the exit threshold $y < -4$, triggering the next-room transition.
- **Died**: player object is removed without a room transition (spike collision, lethal fall).
- **Time-out**: 500 steps elapse without death or completion.

5.3 Checkpoint selection

The default `best.pt` saved during training is the snapshot at the moment the best minimum y is first observed; this is generally an *early*-training snapshot capturing a single lucky episode rather than the converged policy. For evaluation, we instead select the latest training checkpoint (at episode 5,000 for fixed-budget methods, end-of-run for curriculum) as the representative model. We discuss this in Section 7.2.

6 RESULTS

Table 3 summarizes the controlled comparison. Plain DQN with the semantic state encoding achieves the highest completion rate (57-68%), winning by approximately 20 percentage points over the next-best method. Four of the six rows report 0% completion based on our evaluation.

6.1 The perception-fix ablation study

The most consequential ablation study in this work isolates the contribution of the state representation. Holding all other variables fixed: algorithm (DQN), hyperparameters, reward function, action space, network architecture while varying only the encoding of the 9×9 tile grid component of the state, we observe a roughly 8-fold improvement in completion rate (Table 4).

We attribute this gap to the difference in semantic distance between numerically-adjacent tile IDs. Under raw encoding, a spike

Table 4: Perception ablation with the same DQN, hyperparameters, and reward, different tile encoding.

Encoding	Peak completion	First completion (ep)
Raw tile IDs (0-255)	8%	2,600
Semantic (5-class)	**86%**	**1,500**

(ID 17) is numerically as close to a decoration (ID 18) as air (0) is to a solid wall (1). The network must learn to distinguish these classes from gradient signal alone, which it does poorly under our experiments. Regarding the semantic encoding, the same classes are grouped at distinguished values (−1.0 for spikes, 1.0 for solids, 0.0 for air, etc.) and the network learns to navigate immediately.

6.2 Failure-mode analysis

Table 5: Outcome breakdown per method (out of 100 evaluation episodes).

Method	Complete	Died	Timeout
Plain DQN	57	16	27
Dueling + curiosity	37	55	8
Random	0	85	15
BC	0	12	88
Curriculum	0	3	97
Hybrid	0	100	0
DQN with raw IDs	0	12	88

The death/timeout decomposition (Table 5, Figure 3) reveals that different methods fail in numerically different ways:

BC (88 timeouts). The agent locks into a deterministic loop after the first unknown state and runs out of time. This is failure due to covariate-shift of pure imitation [9], where the demonstrations show the optimal trajectory but provide no recovery mechanism for any state slightly off it.

Curriculum (97 timeouts). The same loop behavior, with the agent reaching at most $y = 30$ before stalling. The policy specialized to solve each of the curriculum's intermediate spawn distributions ($y = 15, 30, 50$) and failed to generalize back to the full-level start at $y = 96$, having never seen those early-room states during the stages where it actually learned to complete.

Hybrid (100 deaths). Strikingly worse than random (which had 85 deaths). The agent walks directly into spikes. We trace this to an implementation issue in our expert-buffer construction, where TAS transitions are stored with reward = 0 rather than the true milestone rewards from a replayed episode. With 20% of every gradient update computed against zero-reward expert data, the network is anti-trained on the (state, action) pairs along the optimal trajectory. This pulled the model toward $Q = 0$ on exactly the moves the milestone rewards would teach as positive.

We separately tested a corrected implementation (`hybrid_r3`) that replays the TAS actions through the live environment to capture the actual milestone rewards, fixing the anti-training issue. The

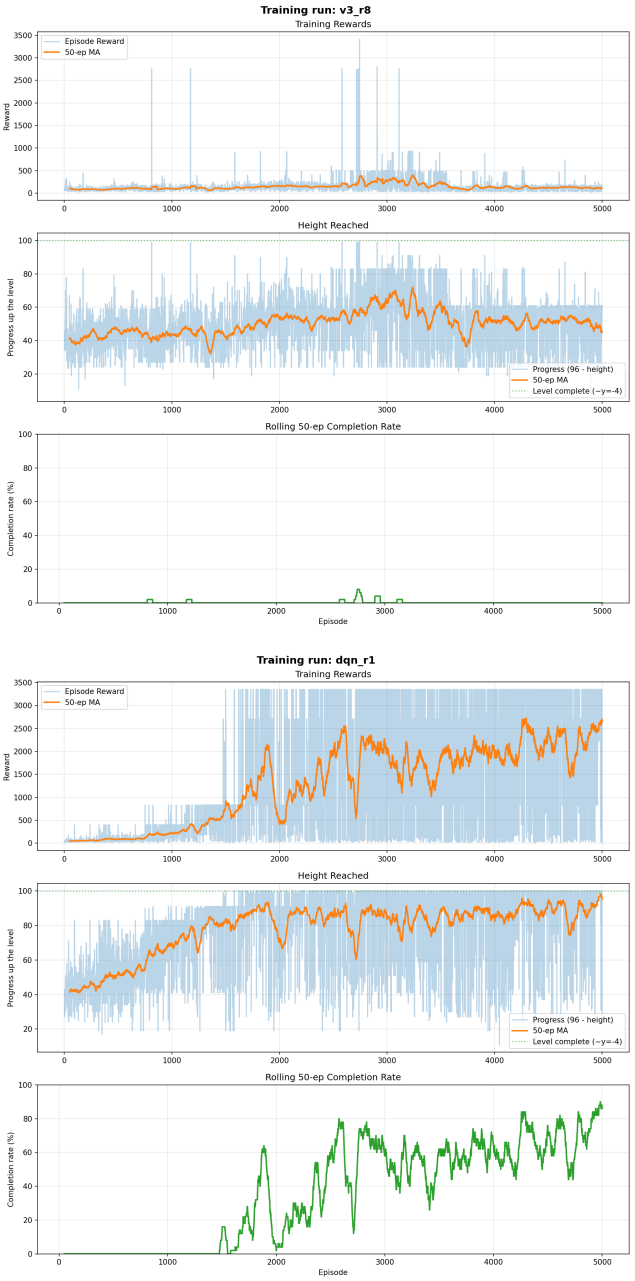

Figure 2: Training curves with raw tile IDs (top, `v3_r8`) and semantic tile encoding (bottom, `dqn_r1`). Same algorithm, hyperparameters, and reward function. The raw-IDs run plateaus and collapse while the semantic-encoding run breaks through at episode 1,500 and sustains 30–60% completion through 5,000 episodes.

corrected run reached $y = 6$ by episode 450, faster than `dqn_r1`'s comparable progress. However, this model subsequently stalled and regressed, where the best height stuck at 6 for 1,400 more episodes while average height climbed from 54 to 72 and per-episode reward

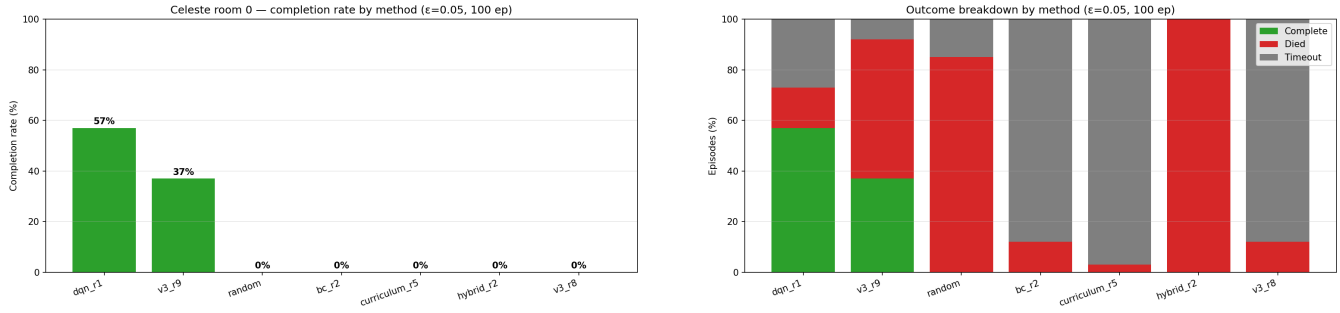

Figure 3: Final comparison across all five methods plus a random baseline ($\varepsilon = 0.05$, 100 episodes per method). Left: completion rate per method, sorted descending. Right: outcome breakdown (complete / died / timeout) for each method. The failure modes diverge meaningfully between methods even when the headline number is the same (e.g., BC and curriculum both report 0% completion but for very different reasons).

fell from 68 to 23. We hypothesize that off-policy bootstrapping divergence caused the failure. The expert reward magnitudes (+100 to +900 per milestone) are 10-50× the online per-frame rewards, the 66 expert transitions are sampled at 20% of every batch, causing the model to overfit. Additionally, the expert demonstrates a path through $y = 6$ on its way to the exit but provides no information other than states the policy has previously encountered and learned from. Neither hybrid implementation beats plain DQN, the failure modes differ but the broader pattern holds. We discuss the implications in Section 7.2.

Plain DQN (16 deaths, 27 timeouts). Mixed failure modes. Most deaths are early ($h \approx 37$–72), suggesting ε-noise pushing the agent into spikes during the climb. Most timeouts are at higher heights ($h \approx 10$–25), suggesting "almost made it" attempts at the upper ledge.

Dueling + curiosity (55 deaths). Dies disproportionately at the upper-ledge zone ($h \approx 22$–25). We hypothesize that the curiosity bonus continues to pull the agent toward unexplored cells even at $\varepsilon = 0.05$, occasionally inducing off-path moves during the final approach.

7 DISCUSSION

7.1 State representation > algorithm choice

Our central observation is that across five methods of this task, every algorithmic enhancement relative to plain DQN with the semantic state encoding either matches or underperforms it. Three of the four 0%-completion variants (BC, Curriculum, Hybrid) are well-motivated in the literature for sparse-reward settings. Thus, their failure in our environment suggests that the bottleneck is not the absence of the technique those methods provide, it is instead related to the agent's perception of the environment.

The 8% to 60% improvement from the perception fix exceeds the contribution of any algorithmic technique we measured. We do not claim this improvement holds outside of Celeste's first room, but suggest it as a methodological prior: *before* reaching for advanced algorithmic techniques (intrinsic motivation, curriculum design, hybrid imitation), examine whether the state encoding actually preserves the semantic structure the policy needs to learn.

7.2 Caveats

Hybrid is brittle in our setting. We evaluated two hybrid implementations. The first (`hybrid_r2`) stores expert transitions with reward = 0, anti-training the network on the optimal trajectory and producing 100% deaths under evaluation. The second (`hybrid_r3`) corrects this by replaying the TAS through the environment to capture true milestone rewards. The corrected run progresses faster than plain DQN initially but stalls at $y = 6$ and regresses, exhibiting off-policy bootstrapping divergence rather than anti-training. Both 0% at deterministic-equivalent evaluation. A robust hybrid implementation would likely require DQfD-style [7] supervised pretraining before online Q-learning, or an annealed expert-influence schedule. These solutions were not implemented as they went outside the scope of this project. Our negative results bound what naive "pre-load the replay-buffer and train normally" hybrids can achieve here, but do not imply that robust hybrid methods may match our beat our plain DQN implementation with additional work.

Dueling-and-curiosity is a 2-axis change. Plain DQN (DQN_R1) and Dueling + curiosity (v3_R9) differ on two axes simultaneously: network architecture (plain MLP vs. value/advantage decomposition) and reward shaping (no curiosity bonus vs. counts-based bonus). Separating the 20-percentage-point gap between them into architectural and reward-shaping components would require an additional run (plain DQN with curiosity bonus, or Dueling DQN without curiosity), which falls under future work plans for this project.

Single-seed comparisons. Each method was run once with one random seed. We observed approximately ±10 percentage points of variance between an isolated 100-episode evaluation of a checkpoint and a batched re-evaluation of the same checkpoint. The ordering of methods is stable across all evaluation conditions we ran, but proper variance estimates would require multiple seeds.

Single-room evaluation. All comparisons are on Celeste's first room. Whether our findings generalize to rooms with different layouts and action requirements (rooms 5+ require dash chains, rooms

12+ have wind, rooms 20+ have falling blocks) is unknown. CelesteBot [6] solves the full game using a genetic algorithm against a Unity re-implementation of the game. This suggests that population-based or on-policy methods with curiosity may be the right algorithmic approach for more complex rooms.

7.3 Future work

- **Robust hybrid implementation.** Our two naive hybrid implementations (one with anti-training, one with off-policy divergence) suggest that simply pre-loading the replay buffer is insufficient in this domain. DQfD-style [7] supervised pre-training before online Q-learning or an annealed expert-influence schedule, are the natural next experiments.
- **Multiple seeds.** Three seeds per method would let us put confidence bands on the comparison and test the significance of the DQN vs. Dueling+curiosity gap.
- **Generalization to other rooms.** The perception fix success is established only for the first room of *Celeste Classic*. Whether it generalizes is unknown.
- **Single-variable architectural ablation.** Run plain DQN with the curiosity bonus to cleanly decompose the DQN/-Dueling + curiosity gap.
- **Larger expert dataset for BC.** 66 transitions is too small to establish whether BC's failure is fundamental (covariate shift) or limited by data.

8 CONCLUSION

We compared five reinforcement learning methods on the first room of *Celeste Classic* under a controlled protocol. Plain DQN with a five-class semantic tile encoding achieved a 57–68% completion rate, and every other method underperformed. The dominant factor was state representation, as replacing raw PICO-8 tile IDs with a semantic encoding raised plain DQN's peak completion from 8% to 86% with no other change. Four of five DQN-family enhancements (Dueling + curiosity, BC, Hybrid, Curriculum) underperformed the baseline once the encoding was fixed. The result suggests that for precise tasks featuring sparse rewards, examining the state representation is at least as important as choosing the algorithm.

CODE AND DATA

Code, pretrained checkpoints, and a debugging dev-log covering all nine iteration phases are available at https://github.com/jmtorr3/celeste-rl.

REFERENCES

[1] Marc G. Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Remi Munos. 2016. Unifying count-based exploration and intrinsic motivation. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[2] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*. 41–48.

[3] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. 2019. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*.

[4] CelesteClassic Community. 2020. Pyleste: A Python implementation of Celeste Classic. https://github.com/CelesteClassic/Pyleste. Accessed: 2026-04.

[5] dav1dm0. 2025. PPO Agent for Pyleste. https://github.com/dav1dm0/PPO-Agent-for-Pyleste. Accessed: 2026-04.

[6] effdotsh. 2024. Celeste-Bot: a genetic algorithm playing Celeste Classic. https://github.com/effdotsh/Celeste-Bot. Accessed: 2026-04.

[7] Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Ian Osband, et al. 2018. Deep Q-learning from demonstrations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.

[8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing Atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602* (2013).

[9] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. 2011. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 627–635.

[10] Salies. 2025. RLeste: a reinforcement learning implementation to solve Celeste Classic. https://github.com/Salies/RLeste. Accessed: 2026-04.

[11] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*.

[12] Hado van Hasselt, Arthur Guez, and David Silver. 2016. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 30.

[13] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. 2016. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. 1995–2003.